

5.6 Set Operations

Union, Intersection and Difference

- Union
 - (subquery) UNION (subquery)
- Intersection
 - (subquery) INTERSECT (subquery)
- Difference
 - (subquery) EXCEPT (subquery)

5.7 Dealing with Duplicates

Duplicates: Set or Bag for your data?

- Set:
 - A set is a collection of distinct elements.
 - No duplicates. Multiplicity does not count.
 - $\{a, a, b\} = \{a, b\}$
- Bag (or multiset):
 - A bag is a collection of multiple instances of distinct elements.
 - Duplicates allowed. Multiplicity counts.
 - $\{a, a, b\} \neq \{a, b\}$
- For a relation stored in your DB: Perhaps no duplicates.
 - Since real-world entities are supposed to be unique.
 - You can use "key" attributes to force uniqueness (e.g. "id" must be different).

Would You Want Duplicates? Sometimes!

- (Intermediate) results may not be distinct entities.
- Their multiplicity may carry information you want.
- Q1: What majors are there?
 - $\{CS, Bio, Econ\}$
- Q2: What major is "most popular"?
 - $\{CS \times 2, Bio \times 1, Econ \times 1\} \rightarrow CS \text{ most popular.}$

SQL: Default about Duplicates

- SELECT-FROM-WHERE: Default is BAG semantics.
 - Why? A query generate results (and not stored relations), the multiplicity probably carries information we want.
- UNION/INTERSECT/EXCEPT: Default is SET semantics.
 - Why? Because these are set operations.
- In either case you can override defaults.

Overriding Default Semantics

- SELECT-FROM-WHERE: Bag semantics
- Override with "DISTINCT"
 - $SELECT \text{ major} \rightarrow \{CS, Bio, Econ, CS\}$
 - $SELECT \text{ DISTINCT major} \rightarrow \{CS, Bio, Econ\}$
- SET operations: set semantics
- Override with "ALL"
 - (subquery) UNION (subquery)
 - (subquery) UNION ALL (subquery)

SQL 5.8 Aggregates

```
SELECT AVG(grade) FROM Enrolls WHERE  
number = "CS411"
```

```
SELECT beer, (AVG(bar), COUNT(price)) FROM Beers, Sells  
WHERE Beers.name = Sells.beer AND brewer = "AB  
InBev"  
GROUP BY beer  
HAVING COUNT(bar) >= 2
```

What is aggregation?

• Aggregate function: E.g. SUM, AVG, MIN, COUNT

- Input: a collection of values
- Output: a value

$\{v_1, \dots, v_n\} \xrightarrow[F]{\text{Aggregate Function}} v$

- Summary of data
- Every element contributes

SQL Aggregation Framework

- 1) FROM-WHERE to generate tuples
- 2) GROUP-BY to organize groups
- 3) Functions to aggregate each group.
- 4) HAVING to filter groups.
- 5) SELECT attributes of groups to output

(33)

1) FROM-WHERE to Generate Tuples

- An aggregate needs to apply upon a collection of tuples.
- Where do these tuples come from?
- We specify these tuples FROM $R_1, \dots, R_m$ WHERE

2) GROUP-BY to Organize Groups

- Aggregates are applied to a collection of tuples.
- What would be "a collection" ^{to} apply to?
- We can specify attributes of tuples to organize them together.
- GROUP-BY $A_1, \dots, A_k$ (subset of attributes from $R_1 \times \dots \times R_m$)
 - Tuples sharing the same $A_1, \dots, A_k$ values will be in the same group.
 - Thus, each group is represented by attributes $A_1, \dots, A_k$.

3) Functions to Aggregate Each Group

- Applying to Attributes
 - SUM, AVG, COUNT, MIN, and MAX.
- Applying to Tuples
 - COUNT(*) counts the number of tuples.

Aggregates: What Members Count?

- Null Values
 - Null does not contribute to aggregation.
 - If there are no non-Null values in a column, then the result of the aggregation is Null (for average, sum) or 0 (for count).
- Duplicates
 - ~~DISTINCT~~ inside an aggregation causes duplicates to be eliminated before the aggregation.
 - E.g. `SELECT COUNT (DISTINCT Price)`

4) HAVING to filter groups

- `HAVING <condition H>`
- The condition refers to "group attributes"
 - GROUP-BY attributes, or
 - Aggregate functions.
- H applies to each group, and groups not satisfying the condition are eliminated.

5) SELECT Attributes of Groups to Output

- `SELECT A1, ..., Ak, F(Ai)`
 - Using only attributes or aggregates of each group.

Attributes of Tuples Vs. Groups

(34)

- In an aggregate query, there are tuples and groups (of tuples).
- They have different sets of attributes.
- `FROM-WHERE-GROUP-BY`: Using attributes of tuples.
- `HAVING` and `SELECT`: Using attributes of groups.

Group-By is Optional: Aggregates without Explicit GROUP-BY

- One group containing all tuples generated from `FROM-WHERE`

```
SELECT AVG(grade) FROM Enrolls WHERE  
number = "CS411"
```

- NO GROUP-BY
 - The group has NO common "group-by" attributes.
- Thus, `SELECT` can only use aggregate functions.